CSC 212: Data Structures and Abstractions

14: Binary search trees (part 2)

Prof. Marco Alvarez

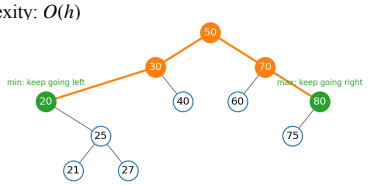
Department of Computer Science and Statistics
University of Rhode Island

Fall 2026



Min and max

- The min is the leftmost node, the max the rightmost
  - complexity: $O(h)$

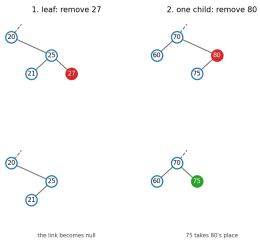



```
Node* BST::min_helper(Node* p) { // p != nullptr
    if (p->left == nullptr)
        return p; // nothing smaller below p
    return min_helper(p->left);
}
```

`max_helper` *is analogous*

Remove: three cases

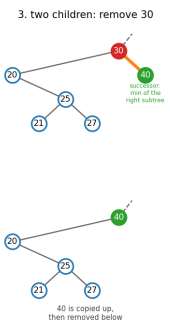
- Find the node `p` with the key, then:
  - **case 1**: `p` is a leaf, delete it, the parent's link becomes `nullptr`
  - **case 2**: `p` has one child, the child takes `p`'s place (delete node)
  - **case 3**: `p` has two children, no single child can take its place (see next slide)
- Time complexity
  - $O(h)$, where h is the height of the tree




```
if root is the node to
remove, update root pointer
after removal
```


Remove: case 3

- Find the **inorder successor**, the smallest key larger than `p`'s (the min of `p`'s right subtree)
  - copy the successor's key to `p`
  - then remove the successor from the right subtree (either case 1 or case 2)
- The in **inorder predecessor**, the max of the left subtree, works just as well



remove_helper

```
void BST::remove(int key) {
    m_root = remove_helper(m_root, key);
}

Node* BST::remove_helper(Node* p, int key) {
    if (p == nullptr)
        return nullptr; // the key is not in the tree
    if (key < p->key) {
        p->left = remove_helper(p->left, key);
    } else if (key > p->key) {
        p->right = remove_helper(p->right, key);
    } else if (p->left == nullptr || p->right == nullptr) { // 1 and 2
        Node* child = (p->left != nullptr) ? p->left : p->right;
        delete p;
        --m_size;
        return child; // nullptr for a leaf
    } else { // 3
        p->key = min_helper(p->right)->key; // copy the successor's key
        p->right = remove_helper(p->right, p->key); // remove it
    }
    return p;
}
```

Same pattern as `insert`*: the returned pointer replaces the old link, so removing the root needs no special case. Cost: one path down to* p*, and one more down to its successor: O(h).*

Q14-1

- In case 3, why is removing the successor always case 1 or case 2, never case 3 again?

A. The successor is always a leaf

B. The successor has no left child: a left child would be a smaller key in p's right subtree

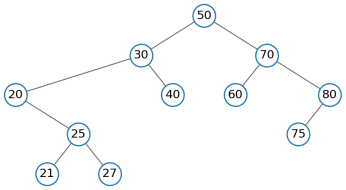
C. The successor has no right child: it is the largest key in its subtree

D. The successor is always p's right child

E. None of the above

Q14-2

- Remove 50 from this tree with `remove_helper`. Which key is at the root afterwards?



A. 40

B. 30

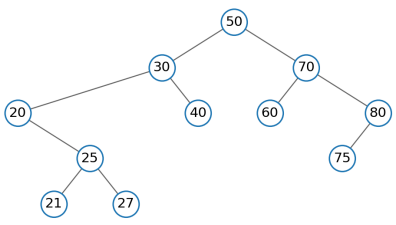
C. 70

D. 60

E. None of the above

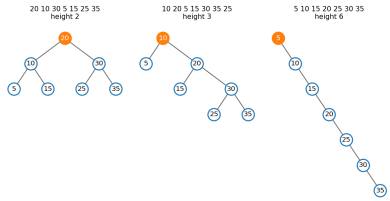
Q14-3

- Remove 27, 40, 80, 20, 30, 50, in this order, using the successor
  - ✓ draw the tree after each removal, and name the case (1, 2, or 3) each one used



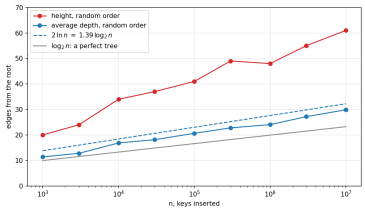
Insertion order

- Insert each sequence into an empty BST. What are the three heights?
  - 20 10 30 5 15 25 35
  - 10 20 5 15 30 35 25
  - 5 10 15 20 25 30 35



The average case

- **Proposition**: if n distinct keys are inserted in *random order* into an empty BST, the average depth of a node is about:
  - $2 \ln n \approx 1.39 \log_2 n$
  - so a random BST is only about 39% deeper than a perfect one



***Average depth**: the mean depth over all nodes, the cost of a typical search*

***Height**: the depth of the deepest node, the cost of the worst search*

Cost of operations

Operation	Best case	Average case	Worst case
search	Θ(1)	Θ(log n)	Θ(n)
insert	Θ(1)	Θ(log n)	Θ(n)
remove	Θ(1)	Θ(log n)	Θ(n)

Animation credit: COS 226: Algorithms and Data Structures, Princeton University

Ordered operations

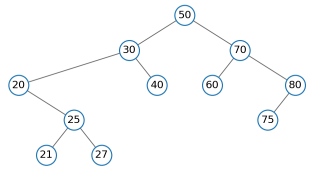
- A BST keeps the keys sorted left to right, so it answers more than *"is it here?"*

Operation	Description	Unsorted vector	Sorted vector	BST
search	is k here?	Θ(n)	Θ(log n)	Θ(h)
insert	add k	Θ(n)	Θ(n)	Θ(h)
remove	delete k	Θ(n)	Θ(n)	Θ(h)
min/max	smallest/largest k	Θ(n)	Θ(1)	Θ(h)
floor/ceiling	closest key <= k / >= k	Θ(n)	Θ(log n)	Θ(h)
rank	how many keys are < k	Θ(n)	Θ(log n)	Θ(h)
range	all keys in [lo, hi]	Θ(n)	Θ(log n + m)	Θ(h + m)

m represents the number of keys reported. Insert into the unsorted vector is linear because it must reject duplicates.

Q14-4

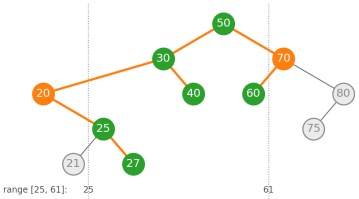
- In this tree, what are `floor(45)` and `ceiling(45)`?



A. 40 and 50
B. 40 and 60
C. 30 and 50
D. 45 and 45
E. None of the above

Range queries

- `range(lo, hi)`: every key in *[lo,hi]*, in sorted order
  - at each node, skip a whole subtree when it cannot hold an answer
  - `p->key <= lo`: the left subtree is all < *lo,* skip it
  - `p->key >= hi`: the right subtree is all > *hi*, skip it



range(25, 61): 6 keys reported (green), 8 nodes visited, 3 never touched.

Cost O(h+m): two boundary paths, plus the m keys in between

Traversals

Visiting every node

- A **traversal** visits every node of a tree exactly once, in a systematic order

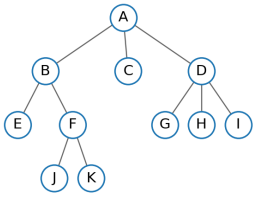
```
algorithm preorder(p) {
    visit(p)
    for each child c of p {
        preorder(c)
    }
}
```

- On a general tree:
  - **preorder:** visit the node, then traverse each subtree, left to right
  - **postorder:** traverse each subtree, left to right, then visit the node

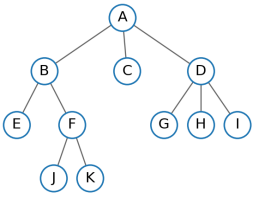
```
algorithm postorder(p) {
    for each child c of p {
        postorder(c)
    }
    visit(p)
}
```

Traversals

preorder: visit, then the subtrees
A B E F J K C D G H I



postorder: the subtrees, then visit
E J K F B C G H I D A



Traversals on binary trees

- Preorder
  - a node before its descendants
  - use case: copying or saving a tree; printing a folder listing
- Inorder
  - a BST's keys in sorted order
  - use case: sorted output, range queries
- Postorder
  - a node after its descendants
  - use case: freeing memory; folder sizes (du); evaluating expressions

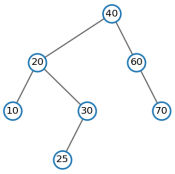
```
void BST::preorder(Node* p) {
    if (p == nullptr) return;
    std::cout << p->key << ' ';
    preorder(p->left);
    preorder(p->right);
}


void BST::inorder(Node* p) {
    if (p == nullptr) return;
    inorder(p->left);
    std::cout << p->key << ' ';
    inorder(p->right);
}

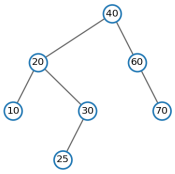

void BST::postorder(Node* p) {
    if (p == nullptr) return;
    postorder(p->left);
    postorder(p->right);
    std::cout << p->key << ' ';
}
```


Traversals on binary trees

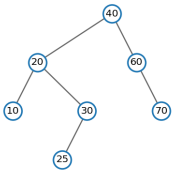
preorder
40 20 10 30 25 60 70



inorder
10 20 25 30 40 60 70



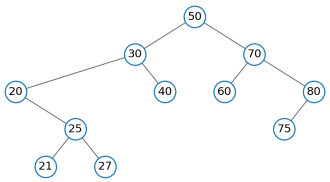
postorder
10 25 30 20 70 60 40



Each traversal is Θ(n)

Q14-5

- Which traversal produces 21 27 25 20 40 30 60 75 80 70 50?

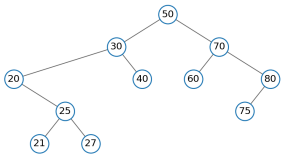


A. Preorder
B. Inorder
C. Postorder
D. Level order
E. None of the above

Q14-6

- Trace this algorithm on the following tree. What does it print?

```
algorithm mystery(p)
    queue q
    q.enqueue(p)
    while not q.isEmpty()
        node n = q.dequeue()
        print(n.key)
        if n.left
            q.enqueue(n.left)
        if n.right
            q.enqueue(n.right)
```



A. 50 30 20 25 21 27 40 70 60 80 75
B. 50 30 70 20 40 60 80 25 75 21 27
C. 20 21 25 27 30 40 50 60 70 75 80
D. 50 70 80 75 60 30 40 20 25 27 21
E. None of the above

Exercises

- Write `preorder` for a ternary tree, whose nodes have the pointers `left, mid, right`
- Write postorder for a *k*-ary tree, whose nodes keep their children in a `std::vector<Node*> children`
- Write `bool is_bst(Node* p)`: is the tree rooted at `p` a BST?
  - checking each node against its children is NOT enough

```
    40
   /  \
  20   60
    \
     45
```

- Write:
  - `Node* BST::floor_helper(Node* p, int key)`
  - `Node* BST::ceiling_helper(Node* p, int key)`